

Architektury systemów komputerowych

Lista zadań nr 10

Na zajęcia 6 i 19 maja 2026

Zadania z tej listy należy rozwiązywać na komputerze z systemem operacyjnym *Linux* dla platformy x86-64. Prowadzący zakłada, że zainstalowana dystrybucja będzie bazowała na *Debianie 11*.

UWAGA! W trakcie prezentacji rozwiązań należy zdefiniować i wyjaśnić pojęcia, które zostały oznaczone **wytluszczoną** czcionką.

Zadanie 1. Poniżej podano zawartość pliku `swap.c`. Wskaż w nim wszystkie wystąpienia definicji i referencji do **symboli** [1, §7.5]. Dla każdego symbolu wskaż jego zasięg widoczności (tj. lokalny, globalny, zewnętrzny) oraz nazwę **sekcji**, w której go umieszczono (tj. `.text`, `.data`, `.rodata`, `.bss`). Wydając polecenie `make swap.o` wygeneruj **plik relokalny** i zweryfikuj swoje odpowiedzi na podstawie wydruku z polecenia `nm`, `objdump` lub `readelf`. Do czego *konsolidator* wykorzystuje tablicę symboli?

```
1 extern int printf(          7 static void incr() {          16 void swap(int i) {
2   const char *, ...);      8   static int count = 0;      17   incr();
3 extern long buf[];         9   count++;                  18   long temp = *bufp0;
4                             10 }                          19   *bufp0 = buf[i];
5 long *bufp0 = &buf[0];     11                             20   buf[i] = temp;
6 static double sum = 0.0;    12 void addf(void) {          21 }
                             13   sum += 3.14;
                             14   printf("sum = %f\n", sum);
                             15 }
```

Zadanie 2. Opisz znaczenie pól rekordu *symbolu* przechowywanego w **tablicy symboli** [2, 4] (czyli pól struktury `Elf64_Sym`, o której można przeczytać w [2, 4-18]). Następnie omów wydruk polecenia `readelf` z opcją `-s` na pliku `swap.o`. Jaka jest pozycja danego symbolu względem początku sekcji?

Co przechowują sekcje `.strtab` i `.shstrtab` [2, 4-17]? Podejrzyj ich zawartość w pliku `swap.o` za pomocą polecenia `readelf` z odpowiednimi opcjami.

Zadanie 3. Poniżej zamieszczono uproszczony wynik kompilacji pewnego pliku z języka C do assemblera. Na podstawie dokumentu *GNU as: Assembler Directives*¹ wyjaśnij znaczenie **dyrektyw assemblera** użytych w poniższym kodzie. Które z dyrektyw przełączają bieżącą sekcję, dopisują zawartość do odpowiednich sekcji, modyfikują informacje przechowywane w nagłówku sekcji lub tablicę symboli?

Opowiedz jak dla tego pliku będą wyglądały zawartości sekcji `.data`, `.rodata`, `.bss`, oraz spróbuj napisać kod w języku C, który mógłby zostać skompilowany do takiej formy.

```
1  .globl foo                14  .globl bar                29  .globl some
2  .data                    15  .bss                      30  .data
3  .align 2                 16  .align 8                    31  .align 32
4  .type foo, @object       17  .type bar, @object        32  .type some, @object
5  .size foo, 2             18  .size bar, 8                33  .size some, 38
6 foo:                     19 bar:                      34 some:
7   .value 314              20  .zero 8                    35  .quad weird
8                           21                          36  .long -3
9  .section .rodata         22  .globl array              37  .zero 4
10 .type abc, @object       23  .bss                      38  .quad abc
11 .size abc, 4             24  .align 32                    39  .quad foo
12 abc:                    25  .type array, @object      40  .string "efghi"
13  .string "abc"           26  .size array, 800
                           27 array:
                           28  .zero 800
```

¹<https://sourceware.org/binutils/docs/as/Pseudo-Ops.html>

Zadanie 4. Rozważmy program składający się z dwóch plików źródłowych:

```

1 /* mismatch-a.c */      1 /* mismatch-b.c */
2 void p2(void);           2 #include <stdio.h>
3                           3
4 int main(void) {         4 char main;
5     p2();                5
6     return 0;            6 void p2(void) {
7 }                        7     printf("0x%x\n", main);
                           8 }

```

Po uruchomieniu program drukuje pewien ciąg znaków i kończy działanie bez zgłoszenia błędu. Czemu tak się dzieje? Skąd pochodzi wydrukowana wartość? Czym różni się **symbol silny** od **słabego**? Zauważ, że zmienna `main` w pliku `mismatch-b.c` jest niezainicjowana. Co by się stało, gdybyśmy w funkcji `p2` przypisali wartość pod zmienną `main`? Co by się zmieniło gdybyśmy w pliku `mismatch-b.c` zainicjowali zmienną `main` w miejscu jej definicji? Czemu dobrym pomysłem jest przekazywanie opcji `-fno-common` do kompilatora?

Zadanie 5. Rozważmy poniższy program składający się z dwóch *jednostek translacji* (czyli dwóch plików). Po uruchomieniu kończy się on z błędem dostępu do pamięci. Przy pomocy debuggera `gdb` zatrzymaj się w miejscu wystąpienia awarii i wyjaśnij jej przyczynę. Gdzie została umieszczona stała znakowa `Hello, world!`? Popraw program tak, by zakończył się poprawnie. Nie wolno modyfikować sygnatury procedury `somestr` i pliku `str-a.c`, ani korzystać z dodatkowych procedur. Gdzie umieszczono ciąg znaków po poprawce?

```

1 /* str-a.c */           5 int main(void) {           1 /* str-b.c */
2 #include <stdio.h>       6     char *s = somestr();       2 char *somestr(void) {
3                           7     s[5] = '\0';               3     return "Hello, world!";
4 char *somestr(void);    8     puts(s);                   4 }
                           9     return 0;
                           10 }

```

Zadanie 6. Omów zaprezentowaną niżej tablicę rekordów relokacji dla sekcji `.text` i `.rodata` z pliku `relo.o`. Zreferuj **proces relokowania** referencji do symboli, dla których asembler wygenerował wpisy relokacji typu `R_X86_64_64` i `R_X86_64_32S` (jest on opisany w [1, §7.7.2]). W trakcie tłumaczenia poniższego kodu na asembler kompilator umieścił tablicę skoków dla instrukcji wyboru `switch` w sekcji `.rodata`. Przyjmij, że w wyniku konsolidacji pliku wykonywalnego zawierającego procedurę `relo3`, została ona umieszczona pod adresem `0x1000`, a tablica skoków pod `0x2000`.

<pre> 1 int relo3(int val) { 2 switch (val) { 3 case 100: 4 return val + 1; 5 case 101: 6 case 103 ... 104: 7 return val + 3; 8 case 105: 9 return val + 5; 10 case 107: 11 return val + 7; 12 default: 13 return val + 11; 14 } 15 } </pre>	<pre> 0000000000000000 <relo3>: 0: 8d 47 9c lea -0x64(%rdi),%eax 3: 83 f8 07 cmp \$0x7,%eax 6: 77 19 ja 21 <relo3+0x21> 8: 89 c0 mov %eax,%eax a: ff 24 c5 00 00 00 jmpq *0x0(,%rax,8) 11: 8d 47 01 lea 0x1(%rdi),%eax 14: c3 retq 15: 8d 47 03 lea 0x3(%rdi),%eax 18: c3 retq 19: 8d 47 05 lea 0x5(%rdi),%eax 1c: c3 retq 1d: 8d 47 07 lea 0x7(%rdi),%eax 20: c3 retq 21: 8d 47 0b lea 0xb(%rdi),%eax 24: c3 retq </pre>
---	--

Oblicz wartości, które należy wstawić w miejsca referencji, do których odnoszą się poniższe rekordy relokacji otrzymane poleceniem `objdump -r`.

```

1 RELOCATION RECORDS FOR [.text]:
2 OFFSET          TYPE          VALUE
3 000000000000000d R_X86_64_32S    .rodata
4
5
6 RELOCATION RECORDS FOR [.rodata]:
7 OFFSET          TYPE          VALUE
8 0000000000000000 R_X86_64_64    .text+0x0000000000000011
9 0000000000000008 R_X86_64_64    .text+0x0000000000000015
10 0000000000000010 R_X86_64_64    .text+0x0000000000000021
11 0000000000000018 R_X86_64_64    .text+0x0000000000000015
12 0000000000000020 R_X86_64_64    .text+0x0000000000000015
13 0000000000000028 R_X86_64_64    .text+0x0000000000000019
14 0000000000000030 R_X86_64_64    .text+0x0000000000000021
15 0000000000000038 R_X86_64_64    .text+0x000000000000001d

```

Zadanie 7 (bonus). Na podstawie [1, §7.12] opisz proces **leniwego wiązania** (ang. *lazy binding*) symboli. Czym różni się **kod relokowalny** (ang. *Position Independent Code*) skompilowany z opcją `-fpic` od kodu nierelokowalnego? Jakie dane przechowują sekcje **procedure linkage table** `.plt` i **global offset table** `.got`? W jaki sposób korzysta z nich *konsolidator dynamiczny*? Czemu sekcja `.got` jest modyfikowalna, a sekcje kodu i `.plt` są tylko do odczytu? Jakie są przewagi *leniwego wiązania* nad *gorliwym wiązaniem*?

Zapoznaj się z obrazkiem [1, 7.19], a następnie zaprezentuj *leniwe wiązanie* na podstawie programu `print.c` z wykładu. Załaduj program do debuggera `gdb` i ustaw **punkt wstrzymania** (ang. *breakpoint*) na funkcję `wypisz`. Po **uruchomieniu** program powinien zatrzymać się we wskazanych miejscach. Zauważ, że za pierwszym wywołaniem procedury `puts` w `.got` jest zapisany inny adres niż za drugim wywołaniem. Pokaż to wykonując program **krokowo**. Gdzie procesor skacze przy pierwszym wywołaniu `puts`?

Literatura

- [1] „*Computer Systems: A Programmer’s Perspective*”
Randal E. Bryant, David R. O’Hallaron; Pearson; 3rd edition, 2016
- [2] „*System V Application Binary Interface*”
<http://www.sco.com/developers/gabi/latest/contents.html>